ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Scala Type Classes — Class Challenge Set, Solutions

Challenge 1 — Encoding JSON objects

```scala
enum Json:
  case JString(value: String)
  case JNumber(value: Double)
  case JBool  (value: Boolean)
  case JArray (elems: List[Json])
  case JObject(fields: List[(String, Json)])
```

(a) Direct encoding

```scala
import Json.*

val example: Json = JObject(List(
  "title"       -> JString("Advanced Programming Languages"),
  "year"        -> JNumber(2026),
  "instructors" -> JArray(List(JString("M. Bugliesi")))
))
```

(b) Monomorphic conversions

```scala
def int2json    (n: Int)          : Json = JNumber(n.toDouble)
def string2json (s: String)       : Json = JString(s)
def bool2json   (b: Boolean)      : Json = JBool(b)
def ints2json   (xs: List[Int])   : Json = JArray(xs.map(int2json))
def strings2json(xs: List[String]): Json = JArray(xs.map(string2json))
```

(c) Movie

```scala
case class Movie(title: String, year: Int, directors: List[String])

def movie2json(m: Movie): Json = JObject(List(
  "title"     -> string2json (m.title),
  "year"      -> int2json    (m.year),
  "directors" -> strings2json(m.directors)
))

// movie2json(Movie("Inception", 2010, List("Christopher Nolan")))
//   ==>
// JObject(List(
//   "title"     -> JString("Inception"),
//   "year"      -> JNumber(2010.0),
//   "directors" -> JArray(List(JString("Christopher Nolan")))
// ))
```

(d) Type class `Jencode`

The cleanest Scala 3 encoding uses **extension methods** so that `x.encode` reads at the call site exactly like `encode x` does in Haskell:

```
trait Jencode[A]:
  extension (a: A) def encode: Json

// ground instances
given Jencode[Int] with
  extension (n: Int)     def encode: Json = JNumber(n.toDouble)

given Jencode[String] with
  extension (s: String)  def encode: Json = JString(s)

given Jencode[Boolean] with
  extension (b: Boolean) def encode: Json = JBool(b)

// the parametric instance: ONE declaration, qualified over the element type
given [A](using Jencode[A]): Jencode[List[A]] with
  extension (xs: List[A]) def encode: Json = JArray(xs.map(_.encode))

// Movie reuses the three field-type instances above
given Jencode[Movie] with
  extension (m: Movie) def encode: Json = JObject(List(
    "title"     -> m.title.encode,
    "year"      -> m.year.encode,
    "directors" -> m.directors.encode
  ))
```

Three points worth noting:

1. **The parametric list instance is a single qualified given.** The clause `(using Jencode[A])` is Scala 3's spelling of the Haskell qualifier `Jencode a =>`. Resolving `List[String].encode` synthesises the dictionary `listJencode[String](using stringJencode)`; resolving `List[List[Int]].encode` synthesises `listJencode[List[Int]](using listJencode[Int](using intJencode))`. The chain is built by the compiler, mechanically.
2. **No `String` vs `List[Char]` clash here.** Unlike Haskell, Scala does *not* identify `String` with `List[Char]`, so there is no need for overlapping-instance machinery: the `Jencode[String]` and `Jencode[List[A]]` instances cover disjoint types.
3. **Method form vs extension form.** A non-extension formulation works just as well:

```
trait Jencode[A]:
  def encode(a: A): Json
```

The given declarations and the parametric instance are written the same way; only the call site changes — `summon[Jencode[Movie]].encode(m)` instead of `m.encode`. The extension form is the idiomatic Scala 3 way to expose type-class operations as if they were methods on the underlying type.

Challenge 2 — Exporting Business Objects

```
case class Customer(name: String, email: String)
case class Invoice (id: Int, amount: Double)
case class Product (code: String, price: Double)

trait Export[T]:
  def toCsv (t: T): String
  def toJson(t: T): String
```

(a) Instances

Each `given` packages the two operations together; CSV is positional, JSON is field-tagged.

```
given Export[Customer] with
  def toCsv (c: Customer): String = s"${c.name},${c.email}"
  def toJson(c: Customer): String = s"""{"name":"${c.name}","email":"${c.email}"}"""

given Export[Invoice] with
  def toCsv (i: Invoice): String = s"${i.id},${i.amount}"
  def toJson(i: Invoice): String = s"""{"id":${i.id},"amount":${i.amount}}"""

given Export[Product] with
  def toCsv (p: Product): String = s"${p.code},${p.price}"
  def toJson(p: Product): String = s"""{"code":"${p.code}","price":${p.price}}"""
```

(b) `exportAll`

The context bound `T: Export` desugars to a `(using Export[T])` parameter. We summon the dictionary once and dispatch on the format string:

```
def exportAll[T: Export](data: List[T], format: String): List[String] =
  val e = summon[Export[T]]
  format match
    case "csv"  => data.map(e.toCsv)
    case "json" => data.map(e.toJson)
    case other  => throw IllegalArgumentException(s"unknown format: $other")
```

Sanity check:

```
exportAll(List(Invoice(1, 99.0), Invoice(2, 150.5)), "csv")
// List("1,99.0", "2,150.5")

exportAll(List(Customer("Alice", "a@x.com"), Customer("Bob", "b@x.com")), "json")
// List("""{"name":"Alice","email":"a@x.com"}""",
//      """{"name":"Bob","email":"b@x.com"}""")
```

Design remark — getting rid of the stringly-typed `format`. The `String` parameter is fragile (typos compile, fail at runtime). A discipline-friendly redesign uses an `enum`:

```
enum Format:
  case Csv, Json

def exportAll[T: Export](data: List[T], fmt: Format): List[String] =
  val e = summon[Export[T]]
  fmt match
    case Format.Csv  => data.map(e.toCsv)
    case Format.Json => data.map(e.toJson)
```

Now the match is exhaustive (the compiler will warn if a future `Xml` case is added and not handled), and unknown formats are rejected statically.